Compliance Reporting & Analytics Module — MVP Specification

Author: Jasfel Analytics

Contents

Compliance Reporting & Analytics Module — MVP Specification 1
1) Purpose 4
2) Scope (MVP) 5
Compliance Reporting 5
Analytics & Dashboards 5
3) Legacy Adapter for Report Generator 6
3.1 Goals 6
3.2 Adapter Pattern 6
3.3 Multi-Tenant Hardening 6
3.4 Refactor Roadmap 7
4) Data Contracts 8
4.1 Canonical Dataset (Input) 8
4.2 ReportArtifact (Output) 8
5) Validation Engine 9
5.1 Architecture 9
5.2 Rule Types 9
5.3 Error Handling 9
5.4 Operator Integration 9
5.5 Testing 9
5.6 Performance 9
5.7 Reliability 9
6) Submission Channels 11
6.1 API Submission 11
6.2 SFTP Submission 11
6.3 Portal Export 11
7) Workflows & UI 12
7.1 Work Queue 12
7.2 Operator Screens 12
8) Analytics & Dashboards 13
8.1 Metrics 13
8.2 Data Pipeline 13

8.3 Dashboards 13
9) Security & Compliance 13
10) Non-Functional Requirements 13
11) Implementation Plan (6 Weeks) 14
11.1 Python Code Roadmap (MVP Trim) 15
Core dependencies 15
Standards, configs, logging, tests, CI 15
Appendix A — Engineering Playbook (Phase 2) 15
Appendix B — NJ Output Layout 16

1) Purpose

Operationalize state charity-care submissions and deliver executive/operational insight. This doc covers two build streams: Compliance Reporting and Analytics & Dashboards, and specifies how to leverage existing state report generator code as a legacy adapter inside a multi-tenant architecture.

Summary
The purpose is to give clinics a reliable way to generate state-approved files and to provide leadership with dashboards that explain what is happening operationally and financially.

2) Scope (MVP)

This MVP covers two main build streams: Compliance Reporting and Analytics & Dashboards. The purpose is to provide FQHCs and hospitals with a working product that ensures charity care submissions are accepted by the state, while also giving leadership insight into operations and patient growth.

Compliance Reporting

- Generate state-ready files in multiple formats (fixed-width, CSV/Excel, XML). Initial focus is NJ plus at least one additional state.
- Tenant-aware validation against state and site policies ensures each clinic's data is properly checked.
- Full audit trail: tracks every step from patient intake to eligibility, appointment, report generation, and final submission.
- Work queue: operators see errors, fix them, and resubmit until the report passes.
- Submission options: API, SFTP, or portal export depending on what the state requires.

Summary
This ensures clinics can generate and submit files the states will accept. It reduces manual work, cuts down on rejections, and provides clear instructions when something needs fixing.

Analytics & Dashboards

- Real-time KPIs for reimbursement, operations, and patient growth.
- Breakouts by tenant, site, program/event, payer, and state allow detailed comparisons.
- Exports (CSV/Excel/PDF) and scheduled email snapshots keep leadership informed.

Summary
The analytics stream gives executives and managers the visibility they often lack today. Instead of waiting weeks for reports, they can see near real-time metrics, compare sites, and spot trends before they become issues.

3) Legacy Adapter for Report Generator

There is legacy single-tenant code available that can be used as a guide to create a new, multi-tenant code base. Jasfel Analytics will treat it as `legacy_report_engine` and wrap it behind a multi-tenant safe interface while progressively refactoring.

3.1 Goals

- **Speed to value**: deliver NJ compliance quickly by leveraging proven code instead of starting from scratch.
- **Safety**: enforce strict tenant isolation and manage secrets properly to avoid any cross-tenant data leakage.
- **Determinism and auditability**: the same input must always yield the same output, with full traceability of how the file was produced.
- **Path to refactor**: gradually peel logic into reusable rules/templates over multiple sprints, reducing dependence on the legacy code.

Summary
The legacy code gives us a head start, but it's not built for multiple clients. Jasfel Analytics will use it as a guide and wrap it so that it's safe, while steadily moving toward a modern, maintainable codebase.

3.2 Adapter Pattern

`ReportAdapter.generate(tenant_id, state_code, run_id, params) -> ReportArtifact` - Loads tenant-scoped config (database connections, extract queries, field mappings). - Invokes the legacy code in a sandboxed, **jailed process** with a per-run temp directory (one per tenant run). Containerization can be added post-MVP if needed. - Produces artifacts: payload file(s), validation log, control totals. - Emits events for audit and dashboards.

Summary
The adapter is like a protective shell around the old code. It feeds in tenant-specific settings, runs the legacy generator safely, and collects outputs in a standard format for the rest of the system.

3.3 Multi-Tenant Hardening

Safeguards required to support multiple FQHCs on the same platform: - Config store (per tenant, per state): connections, queries, field rules, file conventions. - Secrets: managed in Vault/KMS; never stored in plain text. - Isolation: each run uses a **jailed process** with per-tenant temp directories purged after completion. - Limits: apply CPU/memory/timeouts; use streaming I/O for large files. - Observability: structured logs tagged with run_id and tenant_id, metrics collected, error taxonomy applied.

Summary
Multi-tenant hardening is about safety and fairness. Each clinic gets its own protected

space so their data never mixes with another's. If one tenant's file is huge or fails, it won't slow down or affect others. Every run is tracked, logged, and isolated.

3.4 Refactor Roadmap

- Sprint 1–2: Wrap legacy code, externalize configs, add validator, and ship NJ quickly.
- Sprint 3–4: Extract state templates, move transformations to declarative rules (YAML/JSON), add a second state.
- Sprint 5+: Replace remaining custom logic with a rules engine and unit tests per rule.

Summary
Jasfel Analytics won't stay tied to the old code forever. The roadmap makes sure that while NJ goes live quickly, the team steadily moves to a clean, modern system that can scale to more states without fragile custom code.

4) Data Contracts

4.1 Canonical Dataset (Input)

The canonical dataset defines the **input data model** that all reporting and analytics logic will use. It ensures consistency across tenants and states by transforming source EHR/claims data into a standard schema before validation and report generation.

Patient Information - patient_id, last_name, first_name, middle_initial, date_of_birth, gender, ethnicity, race, street_address, city, state, zip, census_tract, migrant_farmer (yes/no)

Eligibility & Coverage - family_size, family_income, payor_source, insurance_name, medicaid_family_care_ever (yes/no), uninsured_family_care_ever (yes/no)

Encounter Information - record_id, visit_date, visit_time, invoice_number, new_patient (yes/no), visit_type (initial/follow-up), icd_1–icd_5, total_charges, total_payment_received, claim_type, uncompensated_visit (yes/no), location_code

Summary
Section 4.1 defines the standard "language" of the system. No matter the tenant's EHR or billing system, data must be transformed into this format. This ensures every downstream module—validation, reporting, analytics—works consistently and can be extended easily for other states.

4.2 ReportArtifact (Output)

This section defines the **NJ fixed-width submission file layout** exactly as provided in the state specification. It lists every field, its byte length, and its absolute positions.

Fixed-width conventions - `X(n)`: alphanumeric, left-justified, space-padded. - `9(n)`: numeric, right-justified, zero-padded. - Dates: `YYYYMMDD` unless otherwise noted. - Blanks: spaces (`X(n)`) or zeros (`9(n)`).

Record Layout The full field list is provided in Appendix B as a Markdown table (NJ Output Layout).

Example

```
ACME HEALTH    ·NJHOSP01      ·20250131·····················...
```

Artifact bundle - Submission file (fixed-width) - Submission manifest (JSON): metadata, checksum, counts - Validation bundle: `validation.json`, `control_totals.json`

Summary
This output contract defines exactly how bytes must be written so the state system ingests files without rejections. Any change requires a rule-set version bump and golden-file update.

5) Validation Engine

The validation engine enforces schema correctness and business rules before files are submitted.

5.1 Architecture

- Stateless service invoked by adapter
- Rules applied in layers: schema, business, cross-record, control totals
- Produces `validation.json` with structured results

5.2 Rule Types

- **Schema rules**: required fields, length, format (dates, numeric)
- **Business rules**: Jasfel-authored, state-specific, versioned
- **Cross-record rules**: uniqueness, totals, duplicates, period boundaries

5.3 Error Handling

- Errors: block submission, must be corrected
- Warnings: flagged, operator acknowledgment required
- Each event carries code, severity, message, field reference

5.4 Operator Integration

- Work queue displays validation events
- Drill-down by field, error code
- Operators fix data and re-run
- AI agents can suggest corrections

5.5 Testing

- Golden-file tests for each state
- Unit tests per rule, edge cases
- Integration tests with canonical dataset

5.6 Performance

- Target: validate 10k records under 10 seconds
- P95 latency ≤ 1 sec per 1,000 records
- Streaming for large files

5.7 Reliability

- Validation jobs complete within 5 minutes for typical files
- Runs are idempotent: same input → same output

Summary

The validation engine ensures reports are correct before submission. It applies schema and business rules, surfaces errors to operators, and maintains deterministic, auditable results.

6) Submission Channels

Reports can be delivered to the state through multiple mechanisms.

6.1 API Submission

- Direct integration where state exposes API
- Secure transport with auth tokens/certs

6.2 SFTP Submission

- Push to state SFTP server
- SSH keys managed per tenant

6.3 Portal Export

- Manual upload via state web portal
- Operators download fixed-width file and upload

Summary
The system supports all state submission methods: API, SFTP, and manual portal upload. This flexibility ensures compliance with diverse state requirements.

7) Workflows & UI

7.1 Work Queue

- Central list of runs with status: Draft, Validating, Errors, Ready, Submitted, Accepted, Rejected
- Filters: tenant, state, date, status
- Actions: view details, fix errors, regenerate, resubmit
- AI agents assist with common fixes, but human operator makes final call

7.2 Operator Screens

- Run details: input config, submission artifacts, control totals
- Validation results: list of errors/warnings with drill-down
- Error drill-down: field-level context and suggestions
- Submission view: files delivered, state response
- Audit log: immutable record of actions

Summary
Workflows and UI let operators manage runs end-to-end. They see statuses, correct errors, acknowledge warnings, and submit files. Every action is logged, with AI assistance available.

8) Analytics & Dashboards

8.1 Metrics

- **Financial**: reimbursement, uncompensated care, payer mix
- **Operational**: validation error rates, first-pass acceptance, turnaround time
- **Growth**: patient counts, new vs returning, site comparisons
- **Quality**: missing data frequency, duplicate rates
- **Predictive**: risk of rejection, forecast of patient growth

8.2 Data Pipeline

- Postgres warehouse with row-level security per tenant
- ETL pulls artifacts into staging tables
- Transforms into star schema (facts, dims)
- Indexes and materialized views for dashboards
- Idempotent loads: same file → same rows

8.3 Dashboards

- **Executive**: KPIs, reimbursement, trends, forecasts
- **Operations**: validation errors, bottlenecks, workload
- **Outreach**: patient funnel, new vs repeat, referral sources
- Future: predictive metrics, AI commentary

Summary
Analytics deliver real-time and predictive insight. Executives see financial trends, operations track error rates, outreach sees patient growth. All powered by an automated ETL pipeline.

9) Security & Compliance

- HIPAA compliance: PHI encrypted at rest and in transit
- RBAC/ABAC: role and attribute-based access control
- Audit logging: immutable event store with run_id, tenant_id
- Data retention: per state/tenant policy, auto-deletion
- Monitoring: intrusion detection, anomaly alerts

Summary
The system is secure and compliant. Data is encrypted, access is controlled, every action logged, and retention enforced automatically.

10) Non-Functional Requirements

- **Performance**: validations < 10s per 10k rows
- **Reliability**: high availability, job retries

- **Scalability**: multi-tenant, handles growing data
- **Security**: HIPAA, secrets in Vault/KMS, RLS in Postgres

Summary
Non-functional requirements guarantee the platform is fast, reliable, scalable, and secure.

11) Implementation Plan (6 Weeks)

Week 1 - Adapter wrapper, tenant config store, NJ dry run (jailed legacy process) - Acceptance: valid NJ file + log

Week 2 - Validation engine v1, operator UI basics, SFTP/manual delivery, audit logging baseline - Acceptance: errors surfaced, resubmission possible

Week 3 - Postgres warehouse + ETL, dashboards v1, error taxonomy integration, alerts - Acceptance: dashboards populate from runs

Week 4 - Add second state template, AI agent scaffolding, operator warning acknowledgment - Acceptance: AI suggestions visible, warnings acknowledged

Week 5 - Security hardening, performance testing, predictive metrics prototype, golden-file suite - Acceptance: P95 validation ≤ 1s/1k rows, golden files reproduce

Week 6 - Pilot tenant test, docs, training, runbooks - Acceptance: end-to-end submission + dashboards live

Summary
Week by week, the system grows from NJ-only reporting into a secure, multi-tenant platform with validation, dashboards, AI support, and predictive metrics. [API, SFTP, manual portal submission]

Summary
Submission channels are the ways reports are delivered. The system supports all required state methods.

11.1 Python Code Roadmap (MVP Trim)

Using the preferred structure:

```
compliance-analytics/
├── app/ (FastAPI, services, adapters)
├── dashboards/ (Dash/Streamlit)
├── data/ (ETL, warehouse)
├── tests/
├── scripts/
├── config/
├── .env
├── requirements.txt
└── README.md
```

Core dependencies

- Python 3.11; FastAPI; SQLModel/SQLAlchemy; Pydantic v2; httpx; Paramiko
- pytest, ruff, black, mypy

Standards, configs, logging, tests, CI

- Type hints, linting, strict typing
- YAML configs validated by Pydantic; secrets via Vault/KMS
- JSON logs + Prometheus counters
- Unit tests, golden-file test (NJ), one integration test
- Simple CI pipeline: lint → unit → integration → build/tag

Summary
This roadmap keeps only the essentials: clean structure, typing/linting, one golden-file, one e2e path, and a simple CI — without Docker.

Appendix A — Engineering Playbook (Phase 2)

Advanced items for post-pilot: performance matrix, supply chain hardening, onboarding templates, advanced analytics, release automation.

Appendix B — NJ Output Layout

Field No.	Reference Number	Field Name	Picture	Byte	Type	From Position	To Position
1	1	Record ID	X(2)	2	X	1	2
2	2	Patient ID	X(12)	12	X	3	14
3	9	Visit Date	X(8)	8	X	15	22
4	99	Visit Time	X(8)	8	X	23	30
5	10	Invoice Number	X(20)	20	X	31	50
6	11	New Patient	X(1)	1	X	51	51
7	12	Patient DOB	X(8)	8	X	52	59
8	13	Patient Gender	X(1)	1	X	60	60
9	14	Payor Source	X(2)	2	X	61	62
10	15	Insurance Name	X(35)	35	X	63	97
11	16	Visit Type	X(2)	2	X	98	99
12	17	Ethnicity	X(1)	1	X	100	100
13	18	Race	X(1)	1	X	101	101
14	19	ICD_1	X(8)	8	X	102	109
15	19	ICD_2	X(8)	8	X	110	117
16	19	ICD_3	X(8)	8	X	118	125
17	19	ICD_4	X(8)	8	X	126	133
18	19	ICD_5	X(8)	8	X	134	141
19	20	Family Size	9(2)	2	9	142	143
20	21	Family Income	9(9)	9	9	144	152
21	22	City	X(20)	20	X	153	172
22	23	Zip	X(5)	5	X	173	177
23	24	Census Track	X(10)	10	X	178	187
24	25	Visit Total Charges	V9(6).99	9	9	188	196
25	26	Total Payment Received	V9(6).99	9	9	197	205

26	27	Claim type	X(1)	1	X	206	206
27	28	Uncompensated Visit	X(1)	1	X	207	207
28	29	Location Code	X(2)	2	X	208	209
29	30	For Medicaid Visit, has this patient ever been under Family Care?	X(1)	1	X	210	210
30	31	For Uninsured, has this patient ever been under Family Care?	X(1)	1	X	211	211
31	32	Migrant Farmer?	X(1)	1	X	212	212

End of Document